

Class 1 — The Physics of Inference

Student Revision Notes · AI Systems Design & Inference Engineering · Mewto Cohort

1. The Big Picture

Everything in inference engineering is downstream of what silicon can physically do. Two numbers govern almost everything: how fast a chip can compute (TFLOPS) and how fast it can move bytes (memory bandwidth, GB/s). Once you know these, most inference behaviour is arithmetic, not magic.

KEY IDEA: Inference in production is not just a forward pass.

It is a full system: network request → auth → queue → scheduler → shared GPU → token stream → metering/billing. The forward pass is ~10% of the engineering.

2. Training vs Serving — Opposite Priorities

Inference happens in both training and serving pipelines, but the goals are opposite:

	Training pipelines	Serving pipelines
Goal	Maximize throughput (samples/sec)	Minimize per-request latency
Batching	Giant, planned batches	Bursty, unpredictable traffic; continuous batching
Key machinery	Checkpointing, gradient sync, optimizers, activation memory, TP/PP	Request scheduling, KV cache, KV memory, quantization for deployment, concurrency
Economics	\$ per token trained	\$ per token served (TTFT + tokens/sec)

KEY IDEA: The KV cache appears only on the serving side. It is the main character of this class.

3. Tokenization — From Text to Tensors

- Models read **integers, not text**. Text is split into subword tokens via BPE or WordPiece (balance of vocab size vs sequence length). Vocab sizes: 32k–128k (Llama 3 at the high end).
- Tokenization runs on the **CPU host**; token IDs are then copied to GPU HBM for embedding lookup — a CPU stage and host-to-device copy sit in your latency budget before the GPU does anything.
- All metering is in tokens: context limits, throughput, and **pricing**. Always count tokens with the model's own tokenizer — tokenizers are not interchangeable.
- Libraries: **tiktoken** (OpenAI, byte-level BPE), **SentencePiece** (Google/Meta, can train tokenizers), **HF Tokenizers** (Rust core, OSS default), plus community ports for token counting.
- Chat models add a **chat template** around your prompt — extra tokens you pay for on every call (seen in the lab: TinyLlama vs GPT-2 token counts differ for the same prompt).

4. The Two Phases of LLM Inference (MOST IMPORTANT)

	Phase 1: Prefill	Phase 2: Decode
What happens	Entire prompt processed in one parallel forward pass; computes Q, K, V for every token; populates the KV cache; produces the first output token	Tokens generated one at a time, auto-regressively; each step feeds 1 new token + attends to all previous tokens by reading the KV cache in every layer
Dominant ops	Large GEMMs (matrix multiplies)	Tiny matmuls + huge reads of weights and KV cache from HBM
Bottleneck	Compute (TFLOPS) — arithmetic throughput, Tensor Core utilization	Memory bandwidth (GB/s) — GPU waits for bytes from HBM
Nature	Parallel	Sequential (cannot parallelize future tokens)
User-facing metric	TTFT (time to first token)	Generation speed (tokens/sec)

KEY IDEA: TTFT = prefill + first decode step. Streaming speed = decode throughput. Two metrics, two phases, two different hardware bottlenecks — you cannot pick a GPU without asking “for which phase?”

Decode speed rule of thumb: $\text{Tokens/sec} \approx \text{HBM bandwidth} \div (\text{model size} + \text{KV cache size per token})$. Example: 7B model @ FP16 = 14 GB; at 2 TB/s bandwidth the ceiling is ~140 tok/s per sequence — before the KV cache eats into it. Extra TFLOPS do not change this.

4.1 Why the phases behave differently: arithmetic intensity

The formal tool behind “compute-bound vs memory-bound” is arithmetic intensity = FLOPs performed ÷ bytes moved. Every GPU has a break-even ratio (peak TFLOPS ÷ bandwidth, roughly 300 FLOPs/byte for an H100 at BF16). Above it you are compute-bound; below it, memory-bound (the roofline model).

- **Forward-pass FLOPs** $\approx 2 \times \text{parameters per token}$. Prefill runs this for hundreds/thousands of tokens against ONE read of the weights → high intensity → compute-bound. Decode runs it for exactly one token against the SAME full read of weights + cache → intensity $\approx 1\text{--}2$ FLOPs/byte → deeply memory-bound.
- **Batching is the lever:** serving B requests in one decode step reads the weights once for B tokens — B× the arithmetic intensity, nearly B× the aggregate tok/s, for almost free. This is WHY batching works so well in decode, why continuous batching is the single biggest serving win, and why the KV cache (which grows with B) is what caps it.
- **Three performance regimes** (Horace He’s framing): compute-bound (prefill), memory-bandwidth-bound (decode), and overhead-bound (Python/dispatch dominating — why small models can be surprisingly slow, and what torch.compile / CUDA graphs attack).

4.2 Prefill methods in real systems

- **Naive prefill:** whole prompt in one pass. Simple; fine for short prompts; long prompts monopolize the GPU → latency spikes for everyone else.
- **Chunked prefill:** prompt split into fixed chunks (~512–2048 tokens), interleaved with decode steps from other requests (vLLM-style). Keeps streams smooth under long prompts.

- **Prefill/decode disaggregation:** prefill on compute-optimized nodes, decode on memory-heavy nodes, KV cache shipped between clusters. Large-scale serving — the phases have different physics, so at scale they are physically separated.

5. The KV Cache — The Growing Tenant

Three things live in GPU VRAM during inference:

- **Model weights** — dominant but fixed; never grow.
- **Activations** — transient; vanish after each forward pass.
- **KV cache** — persistent, and the only one that grows over time.

Why prefill is fine but decode hurts: in prefill the cache is **created once** in one parallel pass; in decode it **accumulates and persists** — every generated token appends new K/V tensors per layer, per head.

KV cache size grows linearly with: (prompt tokens + generated tokens) × layers × head dimension × batch size. The batch-size multiplier is the production killer: 100 concurrent long chats = 100 separately growing caches. This caps concurrency and evicts requests.

- **Concrete scale:** Llama 3 8B @ FP16 caches ~0.5 MB per token (32 layers × 2 tensors × 8 KV heads × 128 head_dim × 2 bytes ≈ 131 KB/layer-pair math folded in). One 8k-token conversation ≈ 4 GB of cache — a quarter of the model's own weight footprint, for ONE user.
- **What engines do when cache runs out:** preempt/evict a request (recompute or swap its cache later), refuse admission, or fragment memory trying — naive contiguous pre-allocation wastes 60–80% of KV memory, which is exactly the problem PagedAttention solves with OS-style paging (non-contiguous fixed-size blocks, near-zero waste).
- **Prefix caching:** shared prompt prefixes (system prompts, few-shot examples, chat history) can be cached once and reused across requests — this is what API providers' "cached input" discount prices, and what SGLang's RadixAttention automates.

5.1 The optimization landscape (know which phase each targets)

Optimization	Targets	Phase
Paged Attention	VRAM efficiency	Decode
Weight quantization	VRAM footprint	Prefill + Decode
KV quantization	VRAM footprint	Decode
Sliding window	VRAM cap	Decode
Speculative decoding	Latency	Decode
Continuous batching	Throughput	Prefill + Decode

KEY IDEA: 4 of 6 famous optimizations target decode. Not a coincidence: decode is memory-bound and its cache grows without limit. First question for any new technique: which phase does it help, and what does it trade away?

5.2 How the big four actually work

- **PagedAttention (vLLM):** treats KV memory like OS virtual memory — cache stored in fixed-size blocks mapped by a block table, so sequences don't need contiguous allocations. Kills fragmentation,

enables cache sharing between sequences (e.g., beam search, shared prefixes). Trade-off: slightly more complex attention kernel.

- **Continuous batching:** instead of waiting for a whole batch to finish (static batching), the scheduler swaps finished sequences out and admits new ones at every decode iteration. Anyscale measured up to 23× throughput vs naive batching, with LOWER median latency. Trade-off: scheduler complexity; interacts with chunked prefill.
- **Quantization:** FP16/BF16 (2 B/param) → FP8/INT8 (1 B) → INT4 (0.5 B). Halving bytes helps BOTH problems at once: capacity (weights + cache fit) and bandwidth (fewer bytes per decode step → directly faster tok/s, since $\text{decode} \approx \text{bandwidth} \div \text{bytes}$). KV-cache quantization applies the same idea to the cache itself. Trade-off: accuracy loss, kernel support, calibration effort.
- **Speculative decoding:** a small “draft” model proposes k tokens cheaply; the big model verifies them in ONE parallel forward pass (prefill-like, compute-bound — using idle FLOPs decode leaves on the table). Accepted tokens are free latency wins; rejected ones fall back to normal decode. Output distribution provably unchanged. Trade-off: draft model must be well-aligned or acceptance rate tanks.

The question driving 2026: long contexts (128k+) + massive models → KV cache explosion. Attacked from both ends: architectures that shrink decode cost per token (GQA/MQA, sliding-window, MoE) and engines that schedule prefill vs decode efficiently (chunked prefill, disaggregation).

6. Frameworks and Engines

- **Orchestration layer:** PyTorch / JAX — model definition and dispatch. Eager mode for debugging; torch.compile captures the graph to strip Python overhead.
- **Inference runtimes:** each flagship feature maps to a lesson above.

Engine	Primary focus	Key innovation
vLLM	High throughput / multi-tenant	PagedAttention + continuous batching
TensorRT-LLM	Low latency / NVIDIA-native	Deep kernel fusion + in-flight batching
SGLang	Structured output / prefix cache	RadixAttention + fast interpreter
TGI (HF)	Production stability / Rust core	Custom FlashAttention integrations

7. GPU Memory Hierarchy

- **HBM (high-bandwidth memory):** the big spec-sheet number (80 GB on H100). Where weights and KV cache live. Large but relatively slow to reach.
- **SRAM (on-chip):** megabytes, next to the compute units, blazing fast. Compute only happens on data that reached SRAM.

Every op = read HBM → SRAM, compute, write back. The HBM↔SRAM pipe is the bandwidth spec (3.35 TB/s on H100 SXM) — and it is the pipe decode saturates.

KEY IDEA: “Memory-bound” ≠ “out of memory.” Capacity problems (won't fit) → quantize or shard. Bandwidth problems (compute starving for bytes) → move fewer bytes per token: quantize, fuse kernels, batch smarter. Different diseases, different cures.

8. Reading a Spec Sheet

The five numbers that matter: FLOPs (prefill speed) · Memory capacity (weights + cache fit) · Bandwidth (decode speed) · Latency (what users feel) · Energy (watts → dollars).

Datasheet row	H100 SXM	H100 PCIe	L40S
FP8 Tensor Core	3,958 TFLOPS	3,026 TFLOPS	1,466 TFLOPS
BF16/FP16 Tensor Core	1,979 TFLOPS	1,513 TFLOPS	733 TFLOPS
GPU memory	80 GB	80 GB	48 GB
Memory type	HBM3	HBM2e	GDDR6 (ECC)
Memory bandwidth	3.35 TB/s	2.0 TB/s	0.864 TB/s
NVLink	900 GB/s	600 GB/s	none
PCIe	Gen5, 128 GB/s	Gen5, 128 GB/s	Gen4, 64 GB/s
Max TDP	up to 700 W	300–350 W	350 W
MIG	up to 7 @ ~10 GB	up to 7 @ ~10 GB	not supported

- **Row → phase mapping:** TFLOPS rows = prefill/TTFT. Bandwidth row = decode/streaming (note the 4× SXM-vs-L40S gap most buyers skim past). Memory = concurrency cap. NVLink = multi-GPU viability (L40S can't play). MIG = multi-tenancy.
- **Read the footnotes:** TFLOPS quoted “with sparsity” → halve for dense workloads. Datasheets are marketing documents.
- **How to choose:** workload first, then the matching row. Long-conversation chatbot (decode-heavy) → bandwidth, then capacity. Offline batch summarization (prefill-heavy, latency-tolerant) → TFLOPS.

8.1 Worked deployment math (memorize the method)

- **Llama 3 8B, BF16, A100 80GB:** weights 16 GB → ~62 GB free for KV. At ~0.5 MB KV/token → ~124,000 tokens of cache in flight. Comfortable; big batches fine.
- **Llama 3 70B, BF16, 2× A100 80GB (tensor parallel):** weights 140 GB = 70 GB/GPU → only ~8 GB/GPU free for KV. With GQA (~0.33 MB/token: 80 layers × 2 × 8 KV heads × 128 dim × 2 bytes) → only ~48,000 tokens across both GPUs. Tight.

KEY IDEA: At scale you buy GPUs for their MEMORY, not their FLOPS — extra GPUs exist to make room for KV cache.

8.2 Why architecture matters: MHA vs GQA vs MQA

- **MHA (e.g. GPT-2):** KV per token = layers × 2 × num_heads × head_dim × bytes — every query head has its own K/V.
- **GQA (e.g. Llama, TinyLlama):** KV per token = layers × 2 × num_kv_heads × head_dim × bytes — few KV heads shared by many query heads. Llama 3 8B: 32 Q heads, 8 KV heads → 4× smaller cache → 4× more concurrent requests on the same HBM.
- **MQA:** a single shared KV head — even more dramatic reduction.

KEY IDEA: GQA was adopted for MEMORY savings, not compute savings — and memory savings translate directly into serving throughput.

9. Software Techniques Born from the Memory Hierarchy

- **Kernel fusion:** unfused Bias+ReLU+Add = N reads + N writes to HBM (intermediates round-trip). Fused into one kernel = 1 read + 1 write; intermediates stay in SRAM. Saved bytes = saved time on memory-bound decode. Tooling: OpenAI Triton (Python-like DSL for fused kernels).
- **FlashAttention — fusion's greatest hit:** fuses the entire attention computation ($QK^T \rightarrow \text{softmax} \rightarrow \times V$) into one kernel, tiling it so the big attention matrix never materializes in HBM. Same FLOPs, far less memory traffic $\rightarrow$ 2–4 $\times$ wall-clock speedup and memory that scales linearly (not quadratically) with sequence length. This is why every serving engine ships a FlashAttention variant.
- **NVIDIA stack:** CUDA runtime (memory alloc, kernel dispatch, sync) $\rightarrow$ cuDNN/cuBLAS (tuned primitives & GEMM — what hits peak TFLOPS in prefill) $\rightarrow$ fusion practice $\rightarrow$ Triton on top. The software stack is NVIDIA's real moat.
- **Scaling out:** NVLink up to 900 GB/s point-to-point (bypasses PCIe); NVSwitch = all-to-all in a node. Tensor parallelism splits layers across GPUs (chatty — needs NVLink); pipeline parallelism puts different layers on different GPUs (less chatty, adds bubbles). Scoreboard: MFU (model FLOPS utilization) — communication is where MFU dies.
- **Beyond CUDA:** AMD ROCm (open-source; HIP ports CUDA code; MI300 leads with HBM capacity + bandwidth — aimed straight at decode/KV cache). Apple MLX (M-series native, unified memory; great for local prototyping).

10. The Economics — Why Output Tokens Cost More

- APIs bill input and output tokens at different rates because **input = prefill** (cheap, parallel, compute-bound) and **output = decode** (expensive, sequential, memory-bound).
- GPT-4o list prices: \$2.50/1M input vs \$10.00/1M output — output is **4 $\times$** the price. (gpt-4o-mini: \$0.15 vs \$0.60 — same 4 $\times$ ratio.)
- At scale (lab example: 10,000 req/day, 500 in + 200 out tokens): output is a minority of the tokens but the **majority of the bill**. Decode is expensive in dollars, not just milliseconds.
- System prompts and chat templates bill as input on **every call** — and the “cached input” discount (half price on GPT-4o) exists because a prefix-cached prompt skips prefill compute entirely.
- **Where the 4 $\times$ comes from physically:** an input token is one slice of a huge parallel GEMM at high GPU utilization; an output token requires a full sequential pass over weights + cache at low utilization. The provider's GPU-seconds per token are simply much higher for output.

11. Lab Takeaways (Colab: Prefill vs Decode)

- We manually split inference: prefill = one forward pass with `use_cache=True` returning logits + `past_key_values` (the KV cache as a real object); decode = a loop feeding 1 new token + the cache each step.
- **GPU timing gotcha:** GPUs are asynchronous — always `torch.cuda.synchronize()` (or `xm.mark_step()` on TPU) before stopping a timer. Never trust an unsynced GPU benchmark.
- **Prompt length:** long prompt $\rightarrow$ prefill several times slower; decode tok/s roughly flat (each decode step is 1 token regardless of prompt length).
- **Model size:** GPT-2 (124M) decodes much faster than TinyLlama (1.1B) — decode speed $\approx$ bandwidth $\div$ model size.
- **Profiler:** prefill = few calls, big tensor shapes; decode = many repeated calls with `seq_len=1` but attention spanning the full cache. Compute-bound vs memory-bound, visible in a table.
- **KV cache on/off:** without the cache, every step re-forwards the whole sequence — dramatically slower. Also measured: GPT-2 (MHA, 12 KV heads) caches more bytes/token than TinyLlama (GQA, 4 KV heads shared by 32 Q heads).

12. Formula Sheet

- **TTFT** = prefill time + first decode step
- **Decode tok/s** $\approx$ HBM bandwidth $\div$ (model size + KV cache per token)
- **Weights (GB)** $\approx$ params $\times$ bytes/param (FP16/BF16 = 2 bytes; e.g. 7B $\rightarrow$ 14 GB, 70B $\rightarrow$ 140 GB)
- **KV per token (MHA)** = layers $\times$ 2 $\times$ num_heads $\times$ head_dim $\times$ bytes
- **KV per token (GQA)** = layers $\times$ 2 $\times$ num_kv_heads $\times$ head_dim $\times$ bytes
- **Total KV cache** $\propto$ (prompt + generated tokens) $\times$ layers $\times$ head_dim $\times$ batch size
- **Max tokens in flight** $\approx$ (VRAM – weights) $\div$ KV per token
- **Request cost** = input_tok $\times$ input_rate + output_tok $\times$ output_rate (output rate $\approx$ 4 $\times$ input)
- **Forward-pass FLOPs per token** $\approx$ 2 $\times$ parameters (so prefill FLOPs $\approx$ 2 $\times$ params $\times$ prompt length)
- **Arithmetic intensity** = FLOPs $\div$ bytes moved; compute-bound if $>$ peak TFLOPS $\div$ bandwidth ($\approx$ 300 FLOPs/byte, H100 BF16)
- **Batched decode** $\approx$ B $\times$ aggregate tok/s (weights read once per step for B sequences) — until KV cache for B sequences exhausts VRAM

13. Self-Test Questions

- Why does prefill benefit more from a GPU upgrade than decode? (Parallel large GEMMs vs sequential memory-bound cache reads.)
- A user complains streaming is slow but first token is fast. Which phase, which hardware spec, and two fixes? (Decode; memory bandwidth; quantize weights/KV, kernel fusion, better batching.)
- Why isn't the KV cache a problem during prefill? (Created once in parallel vs accumulating and persisting every decode step.)
- Compute max tokens in flight for a 13B FP16 model on a 48 GB L40S at 0.4 MB KV/token. (26 GB weights $\rightarrow$ 22 GB free $\rightarrow$ $\sim$ 55k tokens.)
- Why did Llama adopt GQA — compute or memory? What's the throughput consequence? (Memory; 4 $\times$ KV reduction $\rightarrow$ 4 $\times$ concurrency per HBM budget.)
- Why do APIs price output tokens $\sim$ 4 $\times$ input tokens? Explain it in GPU-seconds.
- Chunked prefill vs disaggregated prefill: what problem does each solve?
- Why does batching decode requests nearly multiply aggregate throughput, and what eventually stops you from batching more? (Weights read once per step for all sequences; KV cache $\times$ batch size exhausts VRAM.)
- Speculative decoding uses a second model — why doesn't the extra compute slow things down? (Verification is one parallel, prefill-like pass using FLOPs decode leaves idle.)
- Estimate arithmetic intensity of single-sequence decode for a 7B FP16 model and say whether an H100 is compute- or memory-bound there. ($\approx$ 2 FLOPs/byte vs $\sim$ 300 break-even $\rightarrow$ deeply memory-bound.)

14. Further Reading — Recommended Posts & Papers

Roughly in the order you should read them. The first three are the canon for this class.

Core reading

- [Transformer Inference Arithmetic — kippily \(Carol Chen\)](#) — THE classic. Derives latency, KV cache size, FLOPs and memory-vs-compute boundedness from first principles with napkin math. Everything in Sections 4–5 of these notes, done rigorously.

- [Making Deep Learning Go Brrrr From First Principles — Horace He](#) — The compute-bound / bandwidth-bound / overhead-bound framing, with the factory analogy. Explains why fusion (FlashAttention) wins despite equal FLOPs.
- [LLM Inference Performance Engineering: Best Practices — Databricks/MosaicML](#) — Production-grade treatment of prefill vs decode, TTFT, batching, and benchmarking pitfalls, from a team running these engines at scale.

Engines & scheduling

- [vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention](#) — The PagedAttention announcement: KV fragmentation problem, OS-paging solution, 24× throughput claim. Your Week 2 pre-read.
- [How Continuous Batching Enables 23× Throughput — Anyscale](#) — Static vs continuous batching with clear diagrams and benchmarks; the best single explanation of iteration-level scheduling.
- [Mastering LLM Techniques: Inference Optimization — NVIDIA](#) — Broad, well-illustrated survey: batching, attention variants, quantization, speculative decoding, kernel fusion — from the vendor's perspective.

Going deeper

- [Large Transformer Model Inference Optimization — Lilian Weng](#) — Karpathy-endorsed survey connecting distillation, quantization, pruning, and architecture tricks to inference cost.
- [How To Scale Your Model \(Ch. 7: Transformer Inference\) — Google DeepMind](#) — Blog-style textbook chapter on inference rooflines, KV cache math, and disaggregated serving — TPU-flavored but hardware-agnostic thinking.
- [GPU Glossary — Modal](#) — Interlinked dictionary of every hardware term in these notes: SM, HBM, SRAM, warps, occupancy. Keep open while reading spec sheets.

Primary papers (skim the intros)

- [Efficient Memory Management for LLM Serving with PagedAttention \(arXiv 2309.06180\)](#) — The vLLM/PagedAttention paper — the 60–80% KV waste measurement lives here.
- [FlashAttention: Fast and Memory-Efficient Exact Attention \(arXiv 2205.14135\)](#) — IO-aware attention: tiling + fusion so the attention matrix never touches HBM.
- [GQA: Training Generalized Multi-Query Transformer Models \(arXiv 2305.13245\)](#) — Why Llama-class models use grouped KV heads — the memory-for-throughput trade you measured in the lab.
- [Fast Inference from Transformers via Speculative Decoding \(arXiv 2211.17192\)](#) — The draft-and-verify trick, with the proof that the output distribution is unchanged.

15. Glossary

Term	Meaning
Prefill	Parallel processing of the full prompt; computes Q/K/V for all tokens; populates KV cache; compute-bound
Decode	Sequential one-token-at-a-time generation reading the KV cache; memory-bandwidth-bound
TTFT	Time to first token = prefill + first decode step

Term	Meaning
KV cache	Stored keys/values for all prior tokens, per layer; avoids recomputing history each decode step; grows every token
GEMM	General matrix multiply — the dominant prefill operation
HBM / SRAM	GPU main memory (big, 'slow') / on-chip memory (tiny, fast) — every op moves data between them
Memory-bound	Compute starves waiting for bytes (bandwidth problem — not the same as out-of-memory)
MHA / GQA / MQA	Per-head KV / grouped shared KV / single shared KV — decreasing KV cache size per token
Continuous batching	Dynamically adding/removing requests from the running batch to keep the GPU busy
PagedAttention	vLLM's virtual-memory-style KV cache management (VRAM efficiency)
Chunked prefill	Splitting long prompts into chunks interleaved with other requests' decode steps
Disaggregation	Prefill and decode on separate, differently-specced hardware pools; KV cache shipped between them
Kernel fusion	Merging ops into one GPU kernel so intermediates stay in SRAM (fewer HBM round-trips)
MFU	Model FLOPS utilization — fraction of purchased FLOPS doing useful work
MIG	Multi-Instance GPU — slicing one GPU into up to 7 isolated instances (multi-tenancy)
TP / PP	Tensor parallelism (split layers across GPUs) / pipeline parallelism (different layers per GPU)
Speculative decoding	Small model drafts tokens, large model verifies in one parallel pass — cuts decode latency, output unchanged
MoE	Mixture of Experts — routed sub-networks; complicates KV/memory planning
Arithmetic intensity	FLOPs ÷ bytes moved; decides compute-bound vs memory-bound against the hardware's break-even ratio
Roofline model	Performance ceiling as $\min(\text{peak compute}, \text{bandwidth} \times \text{intensity})$ — the chart behind 'which regime am I in?'
FlashAttention	IO-aware fused attention kernel; attention matrix never materializes in HBM; 2–4× speedup at equal FLOPs
Prefix caching	Reusing KV cache of shared prompt prefixes across requests (system prompts, few-shot); basis of 'cached input' pricing
Overhead-bound	Runtime dominated by Python/kernel-launch overhead rather than compute or memory — attacked by torch.compile/CUDA graphs

Take-home: run the notebook on ≥ 2 runtimes, export the HTML report with your reflection (runtime name + “what surprised you about prefill vs decode?”), and submit. Optional head start for Week 2: the vLLM PagedAttention blog post.